

This boilerplate combines your **Deep Navy/Purple/Green** aesthetic with all the logic for **quizzes, scoring, animated rings, and the radar chart.**

To use this, create a new React app, install the three dependencies, and replace your App.js with the code below.

1. Install Dependencies

Run this in your terminal:

Bash

```
npm install lucide-react recharts canvas-confetti
```

2. The Complete "DataAnalyst Academy" Boilerplate (App.js)

JavaScript

```
import React, { useState, useEffect } from 'react';
import { Database, Code, BarChart3, Sigma, Terminal, Trophy, Flame, CheckCircle2,
ChevronRight, Layout, RotateCcw } from 'lucide-react';
import { Radar, RadarChart, PolarGrid, PolarAngleAxis, ResponsiveContainer } from 'recharts';
import confetti from 'canvas-confetti';

// --- CURRICULUM DATA ---
const modules = [
  {
    id: 'sql',
    name: 'SQL Ninja',
    icon: <Database size={20} />,
    concept: "SQL is the foundation of data retrieval. Master JOINS to connect tables.",
    code: "SELECT u.name, o.total \nFROM users u \nJOIN orders o ON u.id = o.user_id \nWHERE o.status =
'completed';",
    questions: [
      { text: "Which JOIN returns only matching rows in both tables?", options: ["LEFT JOIN", "INNER JOIN",
"FULL JOIN"], answer: "INNER JOIN", explanation: "Inner joins are exclusive to matching pairs." },
```

```

    { text: "Which keyword removes duplicates?", options: ["UNIQUE", "DISTINCT", "FILTER"], answer:
"DISTINCT", explanation: "DISTINCT filters out redundant rows." }
  ]
},
{
  id: 'python',
  name: 'Python Slayer',
  icon: <Code size={20} />,
  concept: "Pandas is the core library for data manipulation. Think of it as Excel on steroids.",
  code: "import pandas as pd\n df = pd.read_csv('sales.csv')\n print(df.groupby('category').sum())",
  questions: [
    { text: "What is the primary data structure in Pandas?", options: ["List", "DataFrame", "Dictionary"],
answer: "DataFrame", explanation: "DataFrames are 2D labeled data structures." }
  ]
},
{
  id: 'stats',
  name: 'Stats Wizard',
  icon: <Sigma size={20} />,
  concept: "Statistics allow you to validate if your insights are real or just random noise.",
  code: "p_value = 0.03\n if p_value < 0.05:\n  print('Statistically Significant')",
  questions: [
    { text: "If a P-value is 0.03, is it significant at the 0.05 level?", options: ["Yes", "No"], answer: "Yes",
explanation: "Any P-value below 0.05 is typically considered significant." }
  ]
}
];

```

```
// --- SUB-COMPONENT: ANIMATED PROGRESS RING ---
```

```

const ProgressRing = ({ percentage, color = "#7c3aed" }) => {
  const radius = 15;
  const circ = 2 * Math.PI * radius;
  const offset = circ - (percentage / 100) * circ;
  return (
    <div className="relative flex items-center justify-center w-10 h-10">
      <svg className="w-full h-full -rotate-90">
        <circle cx="20" cy="20" r={radius} stroke="currentColor" strokeWidth="3" fill="transparent"
className="text-white/5" />
        <circle cx="20" cy="20" r={radius} stroke={color} strokeWidth="3" fill="transparent"
strokeDasharray={circ}
style={{ strokeDashoffset: offset, transition: 'all 1s ease' }} strokeLinecap="round" />
      </svg>
      <span className="absolute text-[8px] font-bold text-white">{Math.round(percentage)}%</span>
    </div>
  );
}

```

```
);  
};
```

```
// --- MAIN APP COMPONENT ---
```

```
export default function AcademyApp() {  
  const [activeId, setActiveId] = useState('sql');  
  const [view, setView] = useState('lesson'); // 'lesson', 'quiz', 'dashboard'  
  const [scores, setScores] = useState({ sql: 0, python: 0, stats: 0, excel: 0, tableau: 0, r: 0 });  
  const [streak, setStreak] = useState(1);  
  
  const activeModule = modules.find(m => m.id === activeId) || modules[0];  
  
  const triggerConfetti = () => {  
    confetti({ particleCount: 150, spread: 70, origin: { y: 0.6 }, colors: ['#7c3aed', '#10b981', '#ffffff'] });  
  };  
  
  const radarData = modules.map(m => ({  
    subject: m.name.split(' ')[0],  
    A: (scores[m.id] / m.questions.length) * 100 || 0,  
  }));  
  
  return (  
    <div className="flex h-screen bg-[#0f172a] text-slate-200 font-sans selection:bg-purple-500/30 overflow-hidden">
```

```
      { /* SIDEBAR */  
      <aside className="w-72 bg-[#0f172a] border-r border-white/5 flex flex-col p-6 overflow-y-auto">  
        <div className="flex items-center gap-3 mb-10 px-2 cursor-pointer" onClick={() =>  
setView('dashboard')}>  
          <div className="w-10 h-10 bg-[#7c3aed] rounded-xl flex items-center justify-center shadow-lg shadow-purple-500/20"><Layout className="text-white" /></div>  
          <span className="font-black text-2xl text-white tracking-tighter italic">DA.</span>  
        </div>
```

```
      <nav className="space-y-2 flex-1">  
        {modules.map((m) => {  
          const pct = (scores[m.id] / m.questions.length) * 100 || 0;  
          return (  
            <button key={m.id} onClick={() => { setActiveId(m.id); setView('lesson'); }}  
              className={`w-full flex items-center justify-between p-4 rounded-2xl transition-all ${activeId  
=== m.id ? 'bg-[#7c3aed] text-white' : 'hover:bg-white/5 text-slate-400'}`>  
              <div className="flex items-center gap-3">{m.icon} <span className="text-sm font-bold">{m.name}</span></div>  
              <ProgressRing percentage={pct} color={pct === 100 ? "#10b981" : "#ffffff"} />  
            </button>
```

```

    );
  }}}
</nav>

<div className="mt-6 p-4 rounded-2xl bg-white/5 border border-white/5 flex items-center gap-4">
  <Flame className="text-orange-500 animate-pulse" />
  <div><p className="text-[10px] font-bold text-slate-500 uppercase">Streak</p><p
className="font-black text-white">{streak} Days</p></div>
</div>
</aside>

{ /* CONTENT AREA */}
<main className="flex-1 overflow-y-auto relative
bg-[radial-gradient(circle_at_top_right,_var(--tw-gradient-stops))] from-[#1e1b4b] via-[#0f172a]
to-[#0f172a]">
  <div className="max-w-4xl mx-auto p-10 pt-20">

    {view === 'lesson' && (
      <div className="animate-in fade-in slide-in-from-bottom-4 duration-500">
        <h1 className="text-6xl font-black text-white mb-6 tracking-tight">{activeModule.name}</h1>
        <div className="backdrop-blur-xl bg-white/5 border border-white/10 rounded-[40px] p-10
shadow-2xl mb-10">
          <h3 className="text-2xl font-bold mb-4 text-purple-400 italic">Today's Focus</h3>
          <p className="text-slate-300 text-lg mb-8 leading-relaxed">{activeModule.concept}</p>
          <div className="bg-black/40 rounded-3xl p-8 font-mono text-sm text-purple-300 border
border-white/5 overflow-x-auto shadow-inner">
            <pre>{activeModule.code}</pre>
          </div>
        </div>
        <button onClick={() => setView('quiz')} className="bg-[#7c3aed] hover:bg-[#6d28d9] text-white
px-10 py-5 rounded-[24px] font-bold text-lg shadow-xl shadow-purple-500/20 active:scale-95
transition-all">Start Assessment</button>
      </div>
    )}

    {view === 'quiz' && (
      <QuizEngine
        module={activeModule}
        onFinish={(score) => {
          setScores(prev => ({ ...prev, [activeId]: score }));
          if(score === activeModule.questions.length) triggerConfetti();
          setView('dashboard');
        }}
      />
    )}

    {view === 'dashboard' && (
      <div className="animate-in zoom-in-95 duration-500">

```

```

    <h2 className="text-4xl font-black text-white mb-10 tracking-tight">Your Skills Radar</h2>
    <div className="grid grid-cols-1 md:grid-cols-2 gap-10">
      <div className="bg-white/5 backdrop-blur-md border border-white/10 rounded-[40px] p-8
h-[400px] shadow-2xl">
        <ResponsiveContainer width="100%" height="100%">
          <RadarChart cx="50%" cy="50%" outerRadius="80%" data={radarData}>
            <PolarGrid stroke="#334155" />
            <PolarAngleAxis dataKey="subject" tick={{ fill: '#94a3b8', fontSize: 12, fontWeight: 'bold' }} />
            <Radar name="Skills" dataKey="A" stroke="#7c3aed" fill="#7c3aed" fillOpacity={0.5} />
          </RadarChart>
        </ResponsiveContainer>
      </div>
      <div className="space-y-6">
        <div className="bg-white/5 border border-white/10 rounded-3xl p-8">
          <h3 className="font-bold text-white mb-4 flex items-center gap-2"><Trophy
className="#10b981" /> Achievement Unlocked</h3>
          <p className="text-slate-400 text-sm">Earn 100% in a module to unlock custom badges.</p>
        </div>
        {radarData.some(d => d.A > 0 && d.A < 60)} && (
          <button onClick={() => setView('lesson')} className="w-full bg-white/5 hover:bg-white/10
border border-white/10 p-6 rounded-3xl flex items-center justify-between group transition-all">
            <div className="text-left"><p className="text-white font-bold">Review Weak Areas</p><p
className="text-slate-500 text-xs">Score below 60% detected</p></div>
            <RotateCcw className="text-purple-500 group-hover:rotate-180 transition-transform
duration-700" />
          </button>
        </div>
      </div>
    </div>
  </div>
</div>
);
}

```

// --- QUIZ ENGINE COMPONENT ---

```

function QuizEngine({ module, onFinish }) {
  const [step, setStep] = useState(0);
  const [selected, setSelected] = useState(null);
  const [showFeedback, setShowFeedback] = useState(false);
  const [score, setScore] = useState(0);

  const q = module.questions[step];

```

```

const handleSelect = (opt) => {
  setSelected(opt);
  setShowFeedback(true);
  if(opt === q.answer) setScore(prev => prev + 1);
};

const handleNext = () => {
  if(step < module.questions.length - 1) {
    setStep(step + 1);
    setSelected(null);
    setShowFeedback(false);
  } else {
    onFinish(score);
  }
};

return (
  <div className="animate-in fade-in zoom-in-95 duration-300">
    <div className="mb-10 flex justify-between items-center text-slate-500 font-bold uppercase text-xs tracking-widest">
      <span>Question {step + 1} / {module.questions.length}</span>
      <span className="text-purple-400">Mastery Assessment</span>
    </div>
    <h2 className="text-4xl font-bold text-white mb-10 leading-tight">{q.text}</h2>
    <div className="grid gap-4 mb-10">
      {q.options.map(opt => (
        <button key={opt} disabled={showFeedback} onClick={() => handleSelect(opt)}
          className={`p-6 rounded-[24px] text-left border-2 font-bold transition-all ${
            selected === opt
              ? (opt === q.answer ? 'border-[#10b981] bg-[#10b981]/10 text-white' : 'border-rose-500
bg-rose-500/10 text-white')
              : 'border-white/5 hover:border-purple-500/50 bg-white/5 text-slate-400 hover:text-white'
            }`}
          {opt}
        </button>
      ))}
    </div>
    {showFeedback && (
      <div className="bg-[#1e293b] p-8 rounded-3xl border border-white/5 animate-in slide-in-from-top-4">
        <p className="text-slate-300 italic mb-6">{q.explanation}</p>
        <button onClick={handleNext} className="w-full py-4 bg-white text-[#0f172a] rounded-xl font-black shadow-lg">
          {step === module.questions.length - 1 ? "Complete Quiz" : "Next Challenge"}
        </button>
      </div>
    )}
  </div>

```

```
    })  
  </div>  
  );  
}
```

Next Step

Your app is now fully functional! Would you like me to generate a **LinkedIn summary** or a **GitHub README** that explains the technical architecture of this project to recruiters?